

Qwen3.8-Flash-Next: which quant should YOU run?

9 files · 3 labs · one RTX 5070 Ti (16 GB) · Strata 0.1.40.3 · every number public & reproducible

RTX 5070 Ti 16 GB · Ryzen 9800X3D

Oct 7–10, 2026 · engine pinned 34CDE150

HumanEval+ · tool-eval ×5 runs · five-bugs · llama-benchy · GSM8K/MMLU/IFEval · needle 260K

blue = ISTA-DASLab

orange = UkisAI Swift-1.5

green = AgentionAI Gyro-S

Compiled 2026-10-10

IN A HURRY? The whole study in 6 lines

- **Tool use is where files actually differ.** On 92 agentic scenarios × 5 runs, scores span 84–89 and reliability floors sit up to 18 points below the mean. Best floor: Swift Q2_0 low (Pass^5 78.3). Best arm: Qwen Q2_0 xhigh (mean 89.4, top rating ★★★★★).
- **All nine files are smart at coding.** Top band 95.7–96.3 on HumanEval+ is reachable six different ways — the fight is about **cost**, not ceiling.
- **Swift (UkisAI) wins by thinking less, not answering faster.** Up to 2.8× fewer tokens per solved task → up to **4.5× less wall time** at the same score.
- **Small file = fast tokens.** 66–68 GB files decode at **129–137 tok/s**; the 76–84 GB files crawl at **91–110**. File size beats quant name.
- **Knowledge is the real family gap.** ISTA IQ2_XS: MMLU **83**. Swift Q2_0: **72**. Same coding score (95.7) — very different brains.
- **One-run tool scores lie.** Run it 5× and the floor drops 8–18 points. Most consistent file: Swift Q2_0 low (Pass^5 78.3).
- **Gyro-S is a great model in an engine that can't call tools.** 96.3 HE+ from one 58.5 GB file — but rc1 has no tool support. Biggest gap found.

THE FOUR QUESTIONS — straight answers

What's the fastest?

Swift IQ3_XXS (UkisAI) — 164 coding tasks in **11m56s**. Small files win: the 66–68 GB files decode at **129–137 tok/s**, ~30% faster than the 76–84 GB files on a 16 GB card.

What's best at coding?

Swift IQ3_S (UkisAI) — ties the top score (96.3), best first-try (94.5), best debugger (20/25 hidden assertions). Gyro-S ties the score; Swift gets there in half the time.

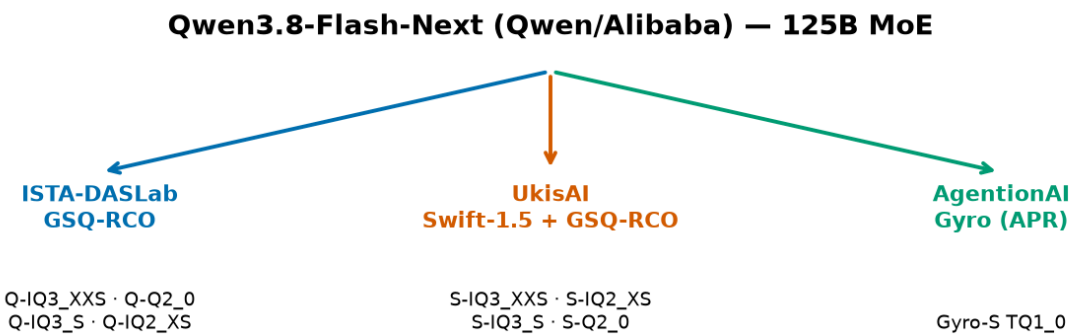
What's best at agentic tools?

Peak: Qwen Q2_0 xhigh and Swift IQ2_XS xhigh — the only ★★★★★ arms (90). Dependable every day: Swift Q2_0 low — highest reliability floor (Pass^5 78.3).

Best overall?

Swift IQ3_S (UkisAI) — ties the top coding score (96.3), best debugger (20/25 hidden), best knowledge in the study (MMLU 85 · GSM8K 98), top-4 reliability floor in agents (Pass^5 76.1), full coding suite in 15m. One file for everything.
! biggest file of the study (83.7 GB) — expert cache on 16 GB

THE CAST — 9 files, one base model, 3 lineages



9 measured files — one base model, three quantization lineages

File	Quant	What it is	GB	Download
● Q-IQ3_XXS ISTA-DASLab	IQ3_XXS	stock quant	75.8	HF repo
● Q-Q2_0 ISTA-DASLab	Q2_0	stock quant	66.4	HF repo
● Q-IQ3_S ISTA-DASLab	IQ3_S	stock quant · daily driver	83.6	HF repo
● Q-IQ2_XS ISTA-DASLab	IQ2_XS	stock quant	67.2	HF repo
● S-IQ3_XXS UkisAI	IQ3_XXS	finetune + quant	76.0	HF repo
● S-IQ2_XS UkisAI	IQ2_XS	finetune + quant · daily driver	68.2	HF repo
● S-IQ3_S UkisAI	IQ3_S	finetune + quant	83.7	HF repo
● S-Q2_0 UkisAI	Q2_0	finetune + quant (experimental)	66.6	HF repo
● Gyro-S AgentionAI	TQ1_0	APR single-file format	58.5	HF repo

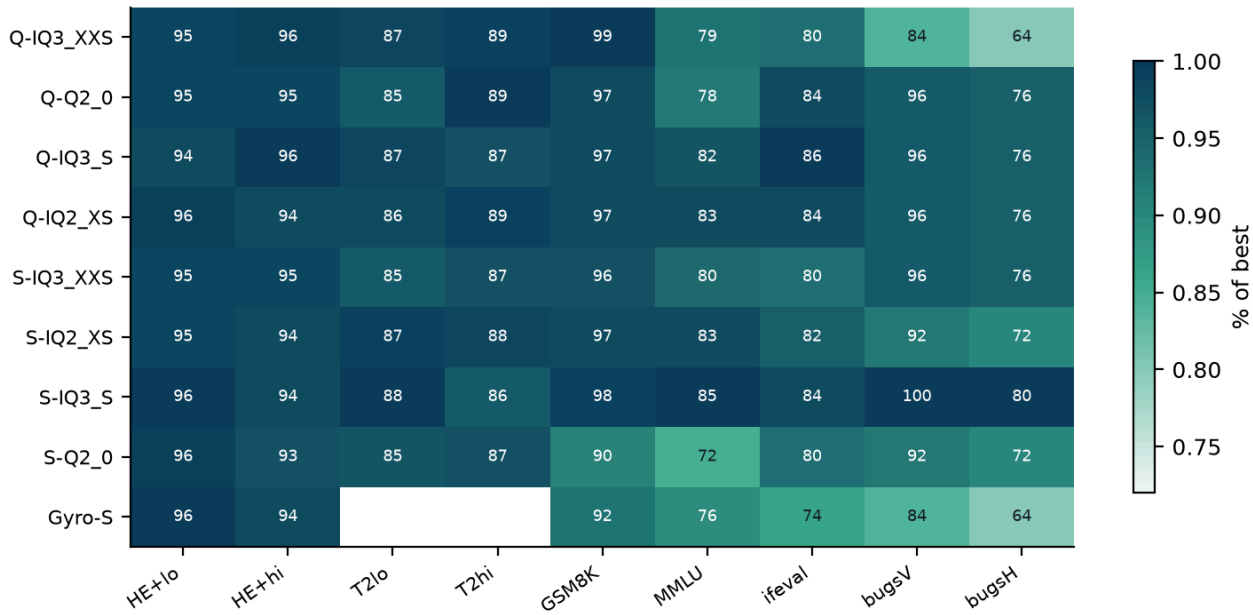
Sources: [ISTA GSQ-RCO quants](#) · [UkisAI Swift-1.5](#) · [AgentionAI Gyro](#) · papers: [GSQ](#) · [RCO](#)

THE SCORECARD — all nine files, one table

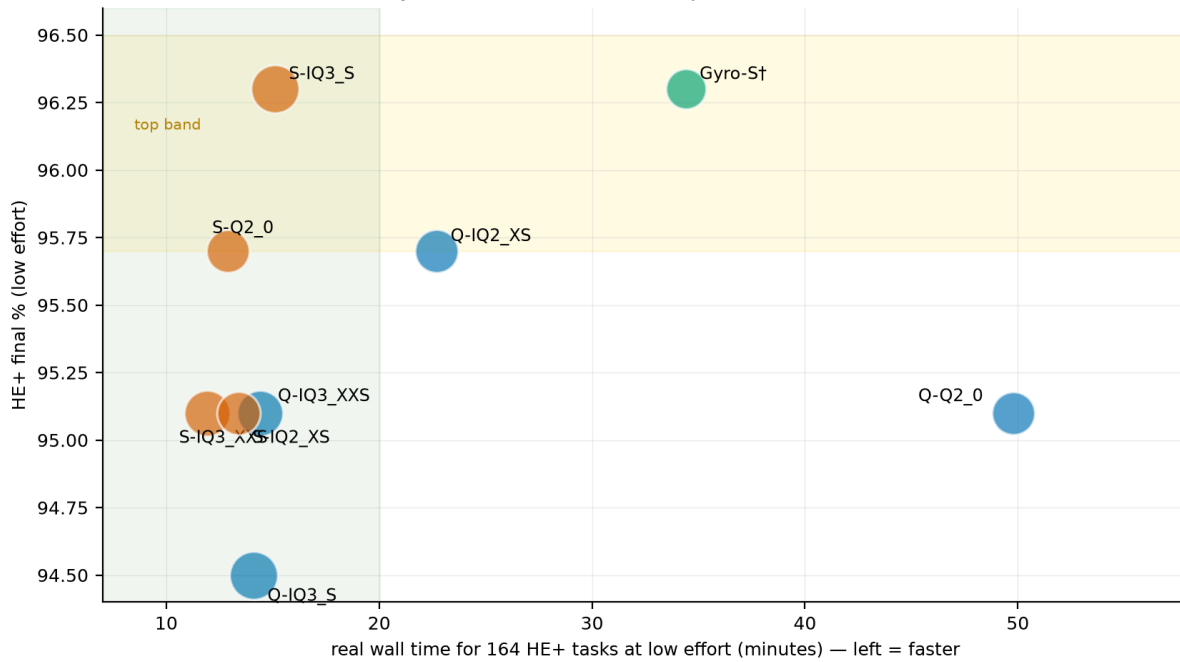
Model (lab)	Tool n=5	Pass^5 floor	HE+ low	HE+ xhigh	GSM8K	MMLU	tok/solve	decode @0K	HE+ wall low
● Q-IQ3_XXS ISTA-DASLab	87.0±2.0 3	76.1 2	95.1	95.7 2	99	79	598.9	110.1	14m24s
● Q-Q2_0 ISTA-DASLab	84.8±0.4	70.7	95.1	95.1 3	97	78	1326.3	136.6 1	49m48s
● Q-IQ3_S ISTA-DASLab	86.8±0.8	73.9	94.5	96.3 1	97	82	526.2 3	90.9	14m06s
● Q-IQ2_XS ISTA-DASLab	86.2±0.8	73.9	95.7 3	94.5	97	83 2	839.4	128.7	22m42s
● S-IQ3_XXS UkisAI	84.6±0.5	75.0	95.1	95.1	96	80	524.8 1	108.7	11m54s 1
● S-IQ2_XS UkisAI	87.2±1.3 2	75.0	95.1	94.5	97	83 3	585.6	129.1 2	13m24s 3
● S-IQ3_S UkisAI	88.2±1.5 1	76.1 3	96.3 1	94.5	98	85 1	525.9 2	92.6	15m06s
● S-Q2_0 UkisAI	85.0±1.0	78.3 1	95.7	93.3	90	72	635.2	128.8 3	12m54s 2
● Gyro-S AgentionAI	—	—	96.3 2	94.4	92	76	657.5	—†	34m24s

1 **2** **3** = best / 2nd / 3rd in that column (for wall time and tok/solve, lower is better) · n=5 = five repeated tool-eval runs · † Gyro decode comes from its own engine log — llama-benchy can't measure it. Every run's raw JSON ships with this PR.

F8 · Dimension grid — darker = closer to the best file in that column



F12 · Quality vs Wall time vs Size — top-left = best; bubble = file GB



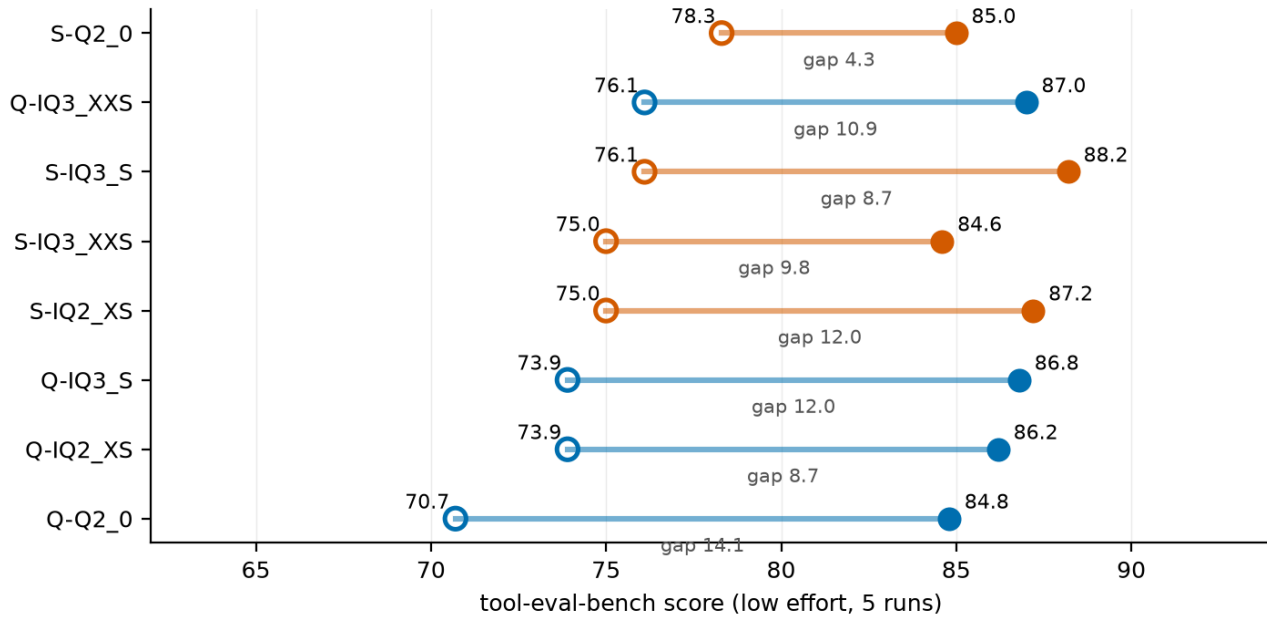
Reading F12: left = less real clock time for all 164 coding tasks (what you actually wait), up = smarter, bubble = file size. Top-left is the dream quadrant — Swift files own it. Gyro-St runs on its own engine; wall measured through the compat tunnel.

RELIABILITY & SAFETY — tool use, run 5 times

Verdict: single runs flatter. Pass⁵ (succeed in **all five** runs) sits 8–18 points under the mean. Safety: every arm flags warnings; low arms flag **more** than xhigh (39 vs 23); 8 of 16 arms are capped at ★★.

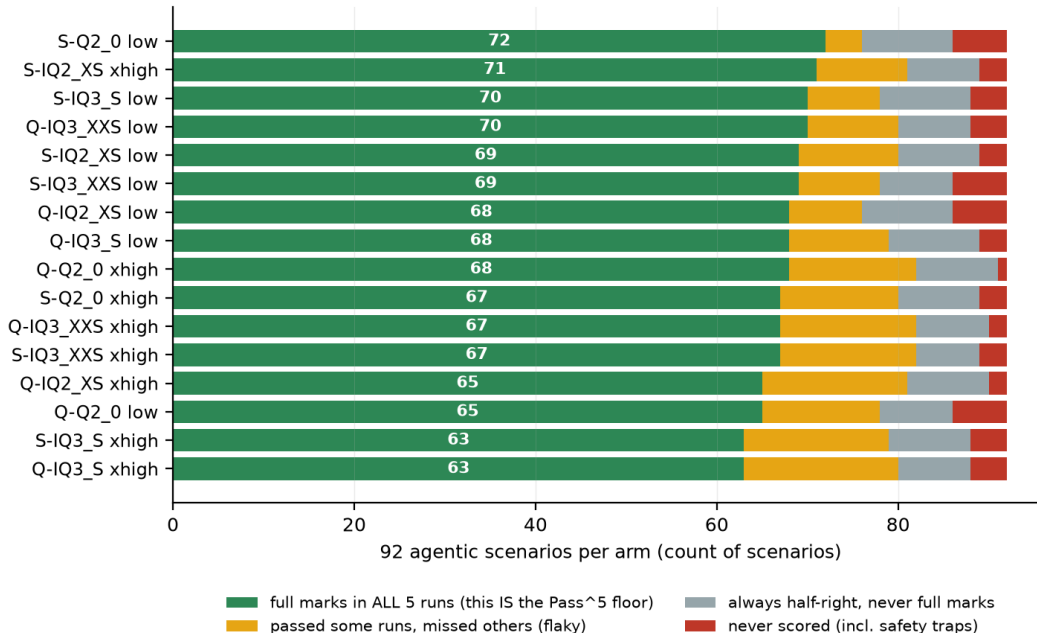
What was run 5 times, and why: the test is **tool-eval-bench** — 92 agentic scenarios (email triage, calendar ops, file/code tasks) where the model must call tools correctly, in order, with valid arguments. Each model ran it **5 identical times at low effort and 5 at xhigh**. Why 5: a single run swings ± 4 points on the same model — one number is noise. **Pass^5** = % of scenarios passed in *all five* runs — your reliability floor, not your lucky day. **Safety warnings** come from scripted traps: e.g. TC-82 sends an email before the recipient is confirmed, TC-84 sends duplicates. Every arm trips at least one trap (the Warnings column is never 0); 8 of 16 trip traps severe enough that the harness caps their rating at **★★★** — severity, not count, decides.

F7 · Reliability — solid dot = average day, open dot = floor (passed ALL 5 runs)



How to read a floor — why a good model scores 68. Each of the 92 scenarios is scored 0 / 1 / 2 (2 = full marks, 1 = half-right). The **mean** averages all that partial credit, so it looks healthy (85–89 for everyone). The **Pass^5 floor** is brutal by design: a scenario counts **ONLY** if the model got **FULL** marks in **ALL 5** runs. Half-right every single time = a fail. So the floor is not "how smart is it" — it is "how many scenarios can your agent trust on the worst day." A model can be brilliant on average and still have a low floor.

F14 · What the reliability floor is made of — green = your agent can trust it daily



Why Swift IQ3_S xhigh floors at 68.5 (the study's lowest). Its mean is fine (85.6) — the collapse is in the floor, and it is a **thinking-effort** effect, not a coding one. At low effort Swift IQ3_S passes 70 scenarios perfectly; at xhigh only 63. Twelve scenarios that were rock-solid at low effort degrade when you ask it to think harder: more reasoning leads it to act boldly — send the confirmation email before the recipient is confirmed, disclose an injected address, report an invoice paid. Those five safety-trap scenarios score **0 in all 5 runs** (TC-84/89/92 = [0,0,0,0,0]), and 16 more flip between runs. xhigh buys it a higher Hard-Mode score and costs it reliability: the model that thinks most is the one that acts most, and acting is where agents get burned.

Arm (model · effort)	Average of 5 runs	Pass^5 floor — RANKED	Fairness gap	Harness rating	Warnings
----------------------	-------------------	-----------------------	--------------	----------------	----------

● S-Q2_0 UkisAI · low	85.0±1.0	78.3 1	4.3	★★★★	5
● S-IQ2_XS UkisAI · xhigh	88.2±1.5	77.2 2	10.8	★★★★★	2
● Q-IQ3_XXS ISTA-DASLab · low	87.0±2.0	76.1 3	10.9	capped ★★★	7
● S-IQ3_S UkisAI · low	88.2±1.5	76.1	8.7	★★★★	4
● S-IQ3_XXS UkisAI · low	84.6±0.5	75.0	9.8	★★★★	4
● S-IQ2_XS UkisAI · low	87.2±1.3	75.0	12.0	capped ★★★	4
● Q-Q2_0 ISTA-DASLab · xhigh	89.4±0.5 1	73.9	15.2	★★★★★	2
● Q-IQ3_S ISTA-DASLab · low	86.8±0.8	73.9	12.0	★★★★	3
● Q-IQ2_XS ISTA-DASLab · low	86.2±0.8	73.9	8.7	capped ★★★	5
● Q-IQ3_XXS ISTA-DASLab · xhigh	88.6±1.1 2	72.8	16.3	capped ★★★	3
● S-IQ3_XXS UkisAI · xhigh	86.6±1.5	72.8	16.3	capped ★★★	4
● S-Q2_0 UkisAI · xhigh	86.8±2.2	72.8	14.2	★★★★	2
● Q-Q2_0 ISTA-DASLab · low	84.8±0.4	70.7	14.1	capped ★★★	7
● Q-IQ2_XS ISTA-DASLab · xhigh	88.6±1.8 3	70.7	17.3	★★★★	1
● Q-IQ3_S ISTA-DASLab · xhigh	86.8±1.3	68.5	18.5	capped ★★★	4
● S-IQ3_S UkisAI · xhigh	85.6±0.5	68.5	17.4	capped ★★★	5
● Gyro-S AgentionAI cannot be tested — rc1 engine has no tool support (C1)					

How to read: rows ranked by the column that matters — Pass^5 floor (score passed in ALL 5 runs). Gold = most dependable arm. green gap = consistent (≤6 pts between average day and floor), red = >12 pts, a model that gambles. capped ★★★ = tripped traps severe enough that the harness caps its rating — 8 of 16 arms. Every arm records warnings (the Warnings column is never 0); warnings alone do not cap you, severity does.

- Most consistent file: Swift Q2_0 low — Pass^5 78.3, gap only 4.3. Best single score: Qwen Q2_0 xhigh 89.4.
- Lowest floors (tied 68.5): Qwen IQ3_S xhigh & Swift IQ3_S xhigh. Both mean ~86 — the floor is low because xhigh makes them act (and trip safety traps), not because they reason worse. Same file at low effort floors 10+ points higher.
- Safety failures are turn-order discipline (email before confirmation, duplicate sends) — harness + prompting fix these more than quant choice does. Effort did NOT make arms safer.

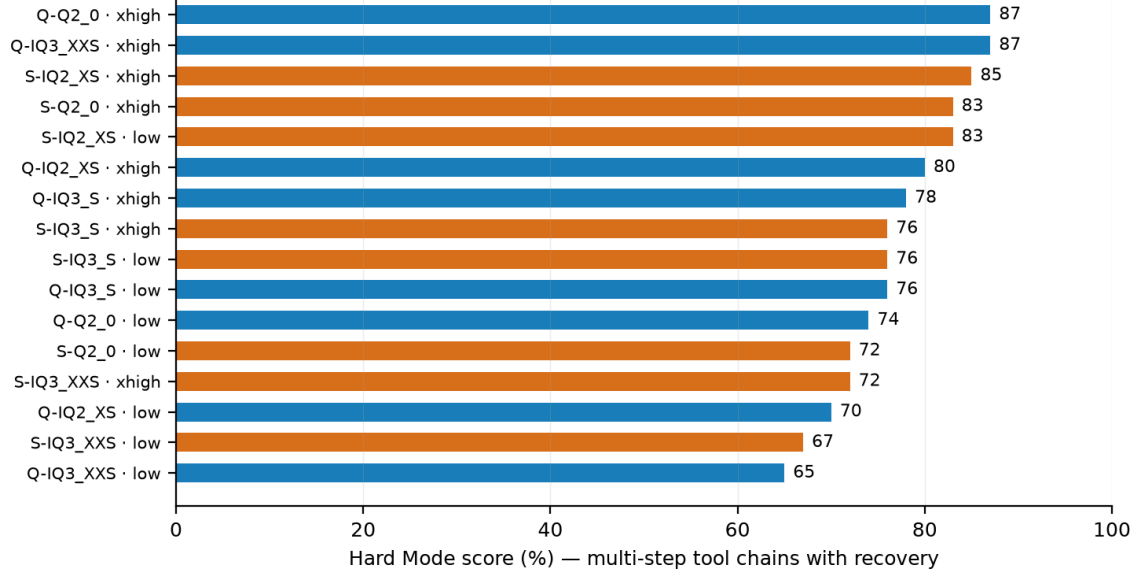
Test: tool-eval-bench — methodology · 5 runs per arm, all raw JSONs public

AGENTIC TOOL USE — 16 arms × 16 categories

Verdict: core tool use (A–D) is at 100 for almost every arm (two low-effort arms dipped to 67 in one category). The fight lives in the edge categories: **P (Hard Mode multi-step) ranges 65→87%**, and effort helps most exactly there (Q2_0: 74 → 87).

Why this chart shows only Hard Mode: tool-eval-bench scores 16 categories (A–P). We measured all of them — and the first ones are **solved by every file**: core skills A–D (single call, parallel calls, ordering) sit at 100 for 30 of the 32 measured category-arms, and category O (structured output under pressure) is pinned at 50 for everyone — a harness ceiling, not a model trait. Plotting all 16 columns gives a green wall with two red stripes: no conclusions to draw. **Hard Mode (P)** is where files actually separate: long multi-step tool chains with recovery from bad tool output. So the chart ranks arms by P — the one column that discriminates.

F10 · The category that separates files: Hard Mode (P)
A-D core tool use ~100 for everyone · O structured output 50 for everyone (harness ceiling)

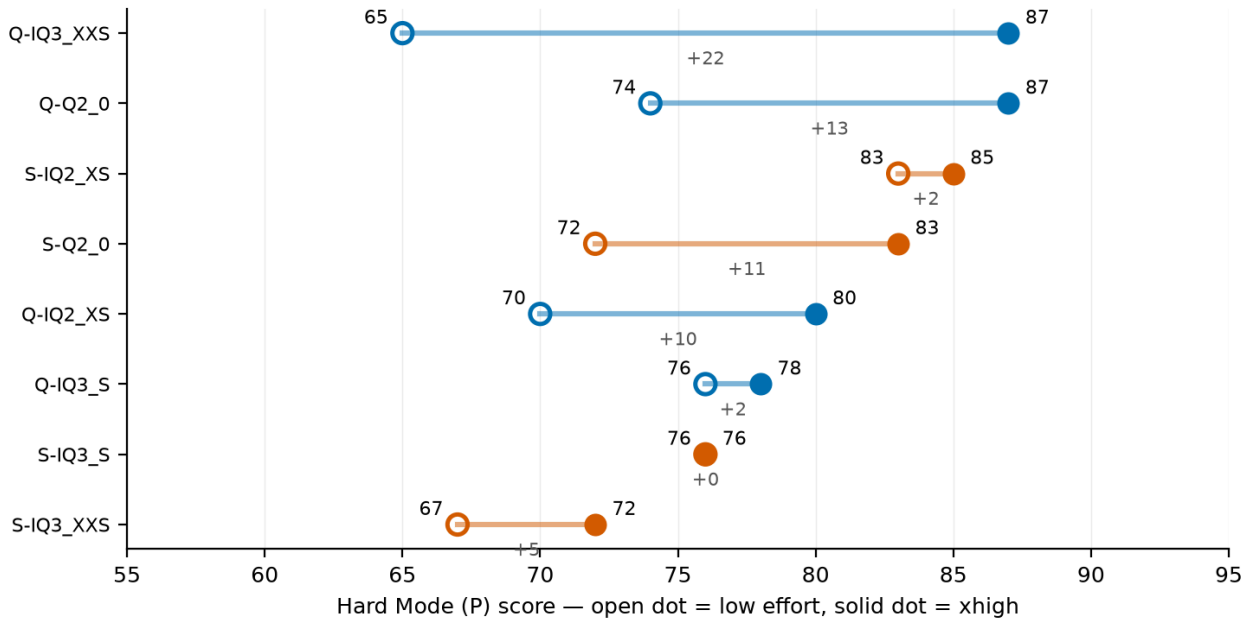


- The spread is real: 65% → 87% on Hard Mode — a 22-point gap between best and worst arm, the widest separation of any test in this study.
- Gyro-S excluded: rc1 engine ignores forced tool_choice and omits tool schemas — it literally cannot call tools (C1).

Does spending more thinking help? On the hardest category, yes.

Reading F13: open dot = Hard Mode score at low effort, solid dot = at xhigh. The line is what extra thinking buys you on multi-step tool chains. Most files gain; a few pay tokens for nothing (see table below).

F13 · Effort buys the hardest category — Hard Mode low → xhigh per file



Every arm, every headline number

Arm (model · effort)	Final score	Rating	Deployability	Responsiveness	Hard Mode (P)	Pass ^Δ 5
Q-Q2_0 ISTA-DASLab · xhigh	90.0 ¹	★★★★★ Excellent	84	71	87	73.9
S-IQ2_XS UkiAI · xhigh	90.0 ²	★★★★★ Excellent	84	69	85	77.2 ²
Q-IQ3_XXS ISTA-DASLab · xhigh	89.0 ³	★★★ Adequate (safety-capped)	82	64	87	72.8
S-Q2_0 UkiAI · xhigh	89.0	★★★★ Good	82	67	83	72.8

● Q-IQ2_XS ISTA-DASLab · xhigh	88.0	★★★★ Good	82	69	80	70.7
● S-IQ2_XS UkisAI · low	88.0	★★★ Adequate (safety-capped)	83	72	83	75.0
● Q-IQ3_S ISTA-DASLab · low	87.0	★★★★ Good	79	60	76	73.9
● Q-IQ3_S ISTA-DASLab · xhigh	86.0	★★★ Adequate (safety-capped)	76	54	78	68.5
● Q-IQ2_XS ISTA-DASLab · low	86.0	★★★ Adequate (safety-capped)	82	71	70	73.9
● S-IQ3_S UkisAI · low	86.0	★★★★ Good	78	61	76	76.1 3
● S-IQ3_S UkisAI · xhigh	86.0	★★★ Adequate (safety-capped)	77	55	76	68.5
● S-Q2_0 UkisAI · low	86.0	★★★★ Good	82	72	72	78.3 1
● Q-IQ3_XXS ISTA-DASLab · low	85.0	★★★ Adequate (safety-capped)	79	64	65	76.1
● S-IQ3_XXS UkisAI · xhigh	85.0	★★★ Adequate (safety-capped)	78	61	72	72.8
● Q-Q2_0 ISTA-DASLab · low	84.0	★★★ Adequate (safety-capped)	82	76	74	70.7
● S-IQ3_XXS UkisAI · low	84.0	★★★★ Good	79	67	67	75.0

- **Effort buys the hardest category:** Q2_0 jumps 74 → 87 and base IQ3_XXS 65 → 87 on Hard Mode going low → xhigh. If you need multi-step agents, spend the tokens there.
- **The only two ★★★★★ arms:** Qwen Q2_0 xhigh and Swift IQ2_XS xhigh (both 90). Eight of sixteen arms are safety-capped at ★★★ — the cap, not the score, is what separates ratings.
- **Deployability vs responsiveness:** xhigh arms answer slower (median turn up to +19%). For chat-style agents, low effort is the honest default; reserve xhigh for multi-step chains.

Tests: [tool-eval-bench categories A–P](#) · [Hard Mode spec](#)

CODE — HumanEval+ 164 tasks

Verdict: the top band is shared — **cost to reach it is not**. Swift owns the three fastest walls (11m56s / 12m56s / 13m23s) at 95.1–95.7. Three arms tie for the top score 96.3: Gyro-S low, Swift IQ3_S low, Qwen IQ3_S xhigh.

What this test is: HumanEval+ = 164 code-writing tasks, each auto-checked against unit tests — no judge, no vibes. **Final** = pass rate after the repair lane (a failed task gets one guided retry); **first-try** = raw one-shot ability. **Wall** = real clock time for all 164 tasks at that effort setting. **low / xhigh** = how much thinking the model is allowed to spend.

11m56s

fastest HE+ run
Swift IQ3_XXS low · 95.1

96.3

top HE+ score (×3 files)
Swift IQ3_S · Gyro-S · Q-IQ3_S xhigh

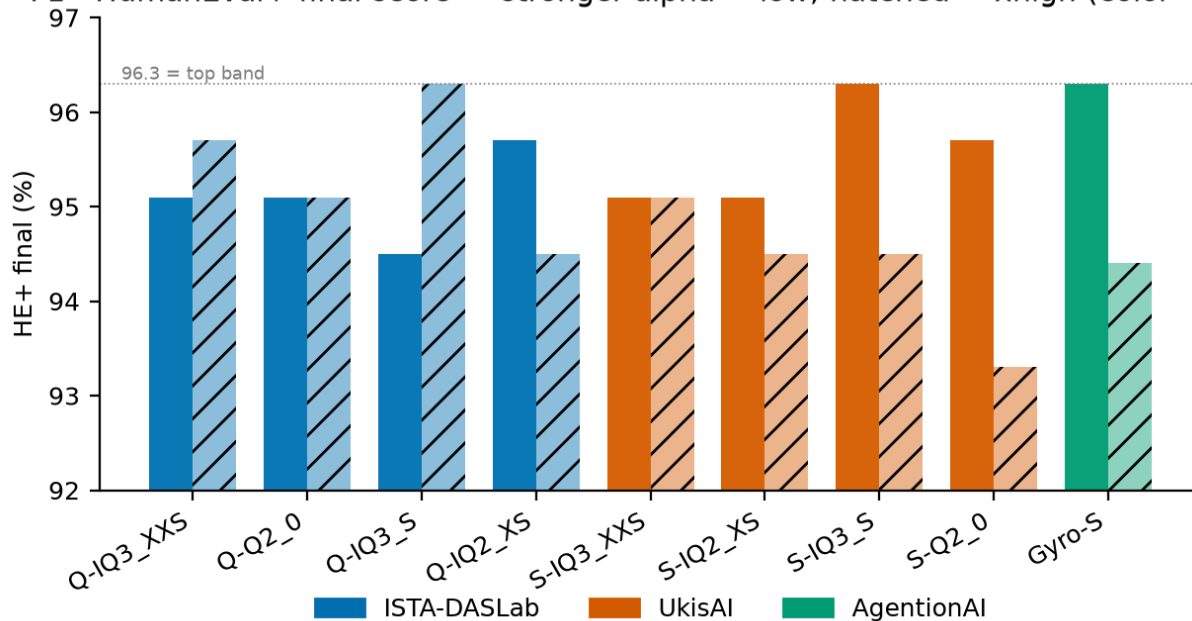
95.7

ISTA IQ2_XS low — at 2-bit size
only 0.6 below the class top

3–11

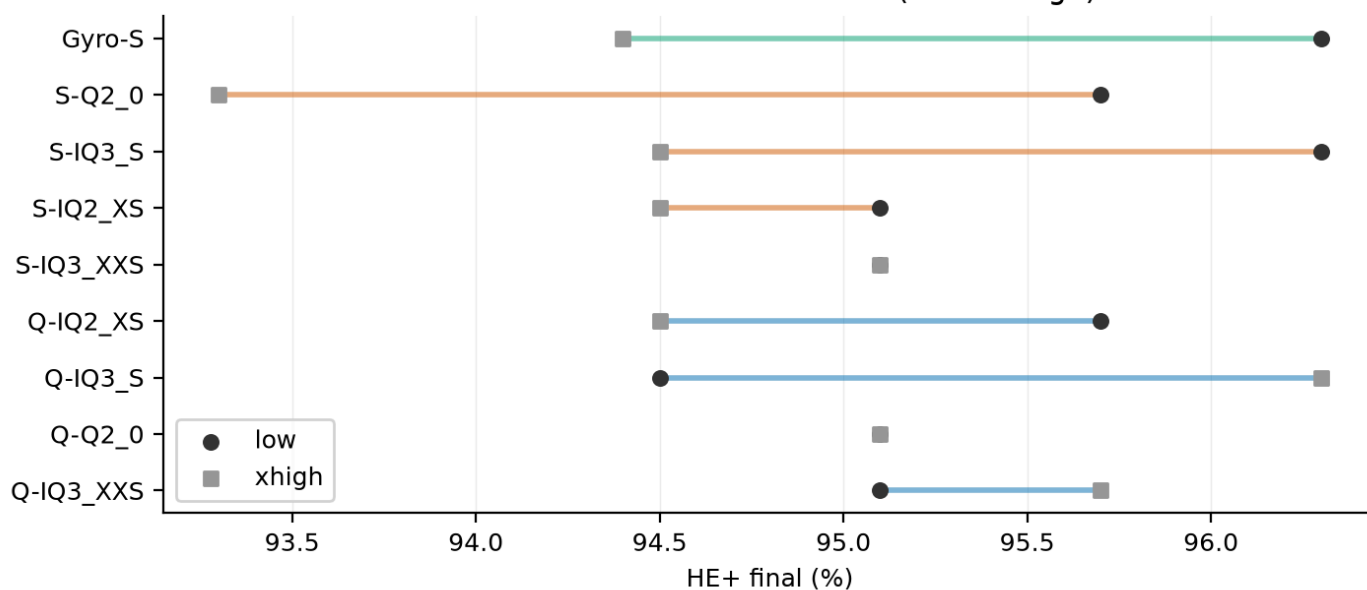
successful repairs per file
the repair lane is real

F1 · HumanEval+ final score — stronger alpha = low, hatched = xhigh (color = lab)



Model (lab)	low final	low first-try	low wall	xhigh final	xhigh wall	effort Δ
Q-IQ3_XXS ISTA-DASLab	95.1	92.6	14m25s	95.7 2	76m16s	+0.6
Q-Q2_0 ISTA-DASLab	95.1	92.7	49m49s	95.1 3	62m17s	+0.0
Q-IQ3_S ISTA-DASLab	94.5	92.7	14m05s	96.3 1	67m18s	+1.8
Q-IQ2_XS ISTA-DASLab	95.7 3	90.9	22m44s	94.5	75m40s	-1.2
S-IQ3_XXS UkisAI	95.1	92.7	11m56s 1	95.1	47m11s	+0.0
S-IQ2_XS UkisAI	95.1	89.6	13m23s 3	94.5	63m04s	-0.6
S-IQ3_S UkisAI	96.3 1	94.5	15m04s	94.5	43m32s	-1.8
S-Q2_0 UkisAI	95.7	90.2	12m56s 2	93.3	56m19s	-2.4
Gyro-S AgentionAI	96.3 2	92.1	34m21s	94.4	329m14s	-1.9

F6 · Effort knob effect on HE+ (low → xhigh)



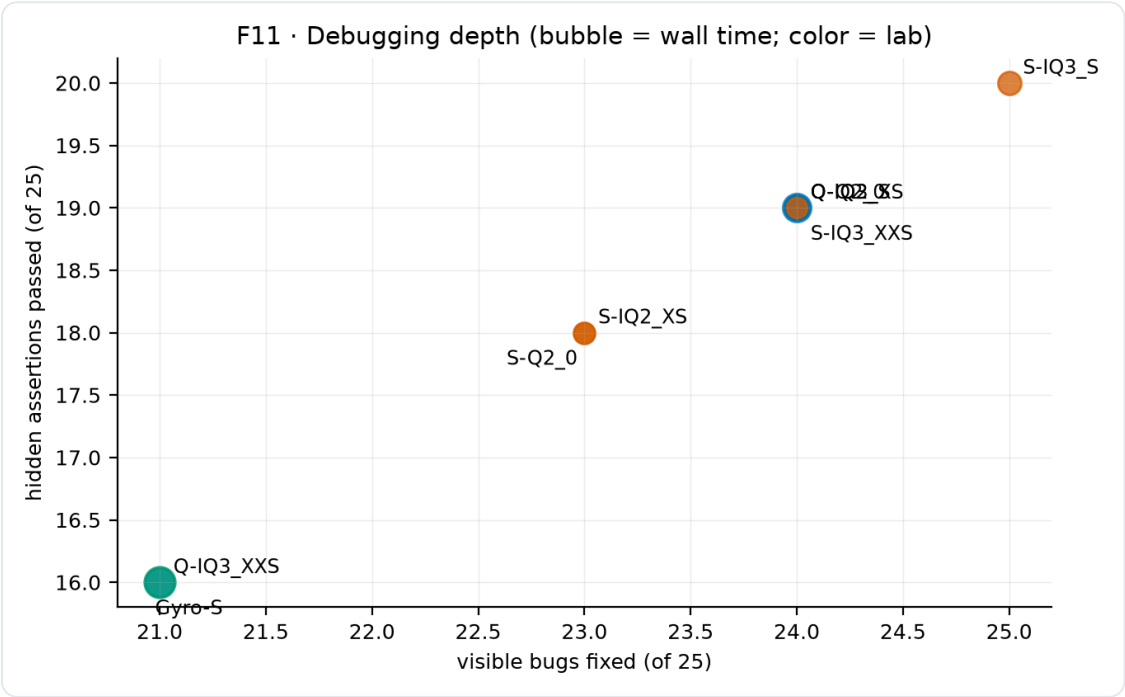
KNOWLEDGE & DEBUGGING — where the families split

Verdict: coding scores hide an **11-point knowledge gap**: ISTA IQ2_XS MMLU 83 vs Swift Q2_0 72 — identical HE+. Debugging depth separates 16→20 hidden assertions; the effort knob does not fix it.

What these tests are: **GSM8K** = grade-school math word problems. **MMLU** = 57-subject multiple-choice knowledge exam. **IFEval** = following explicit instructions (format, length, constraints). **Needle** = hide one fact in a 260K-token document, ask for it. **Five-bugs** = our home-grown debugging test: **5 buggy Python files, each handed to the model 5 times (25 sessions)**. Each session the model returns a fixed file, auto-graded twice: **visible checks** = the obvious bug is gone; **hidden assertions** = edge cases the fix must ALSO satisfy (off-by-one, empty input, ordering). A file scoring 25/25 visible but 16/25 hidden "fixes" bugs superficially — passes what you can see, fails what matters. Closest thing in this study to real maintenance work.

Model (lab)	GSM8K	MMLU	IFEval p/i	Needle	five-bugs visible	five-bugs hidden
Q-IQ3_XXS ISTA-DASLab	99 1	79	80/86	100% @ 254K	21/25	16/25
Q-Q2_0 ISTA-DASLab	97 3	78	84/90	100% @ 254K	24/25 2	19/25 2
Q-IQ3_S ISTA-DASLab	97	82	86/91	100% @ 254K	24/25 3	19/25 3
Q-IQ2_XS ISTA-DASLab	97	83 2	84/90	100% @ 254K	24/25	19/25
S-IQ3_XXS UkisAI	96	80	80/87	100% @ 254K	24/25	19/25
S-IQ2_XS UkisAI	97	83 3	82/88	100% @ 254K	23/25	18/25
S-IQ3_S UkisAI	98 2	85 1	84/90	100% @ 254K	25/25 1	20/25 1
S-Q2_0 UkisAI	90	72	80/87	100% @ 254K	23/25	18/25
Gyro-S AgentionAI	92	76	74/83	100% @ 126K	21/25	16/25

- **Best MMLU:** Swift IQ3_S 85 — then ISTA IQ2_XS & Swift IQ2_XS at 83.
- **Eight files retrieve 100% at 260K** — context reliability is table stakes in this class. Gyro-S's serve caps at 131K; it passes 100% there.
- **Best debugger:** Swift IQ3_S (25/25 visible, 20/25 hidden). Hidden-assertion depth separates 16→20; cranking effort does not fix it.



Tests: [GSM8K](#) · [MMLU](#) · [IFEval \(lm-eval-harness\)](#) · [Needle-in-a-Haystack](#) · five-bugs scenario pack via [Strata](#)

REASONING ECONOMICS — the number vendors hide

Verdict: Swift's edge is **token economy**. Same quality, far fewer thinking tokens → the wall-time gap. xhigh multiplies tokens on most files (Gyro-S: 658 → 2680 per task!) and pays for it in minutes.

What we measured: every HE+ run logs **tokens generated per solved task** and **real wall time**. "Generation share" = how much of the wall is the model actually producing tokens (vs harness overhead). This is where "it thinks a lot, so it's slow" becomes a number you can compare.

2.8×

fewer tokens per solved task
Swift IQ3_S (526) vs Q-IQ3_S (1490)

4.2×

less wall time at equal quality
95.1 score: 11m56s vs 49m47s

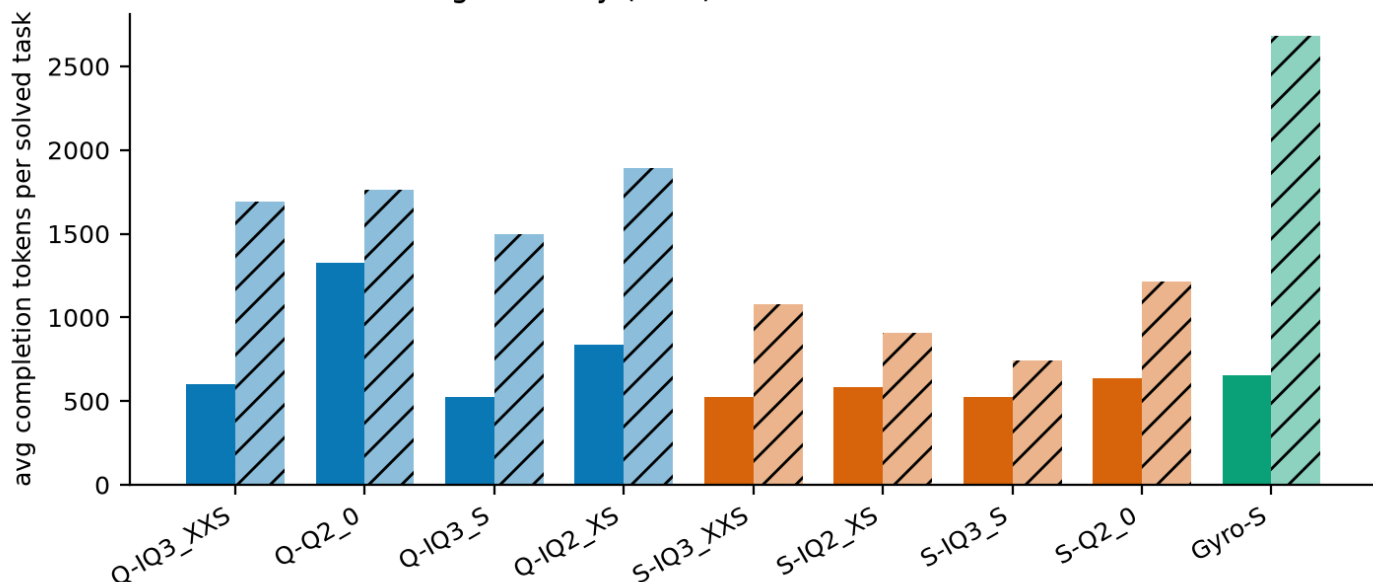
70–97%

of HE+ wall is generation
token discipline is the lever

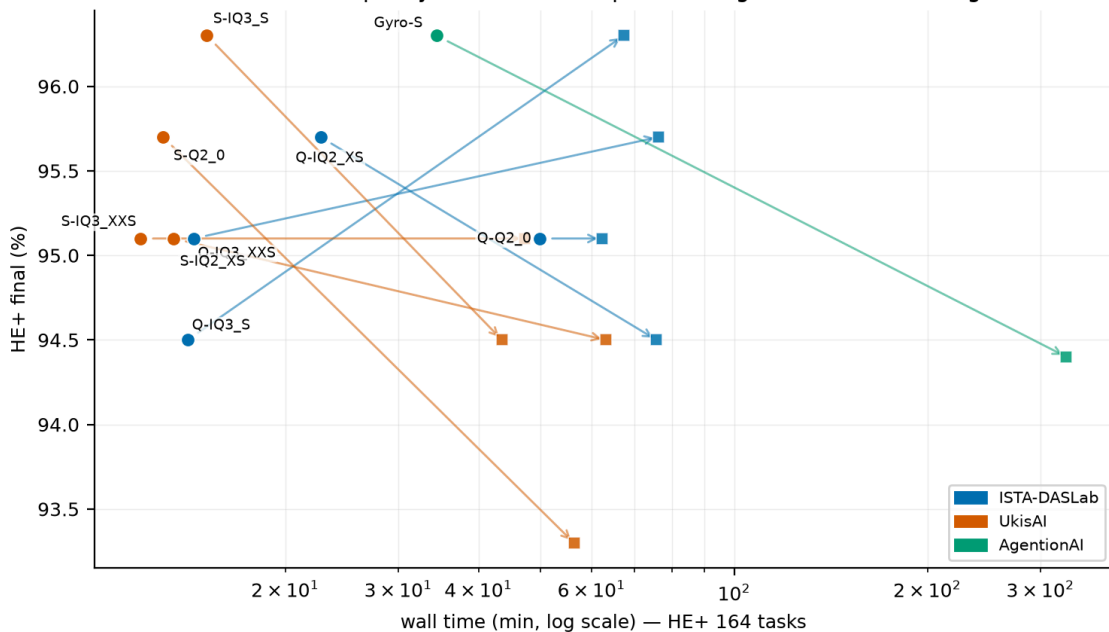
2680

tokens/task for Gyro-S xhigh
the most expensive thinking in the study

F3 · Reasoning economy (HE+): tokens to solve — Swift files sit low



F2 · Time-to-quality: circle = low, square = xhigh, arrow = low→xhigh

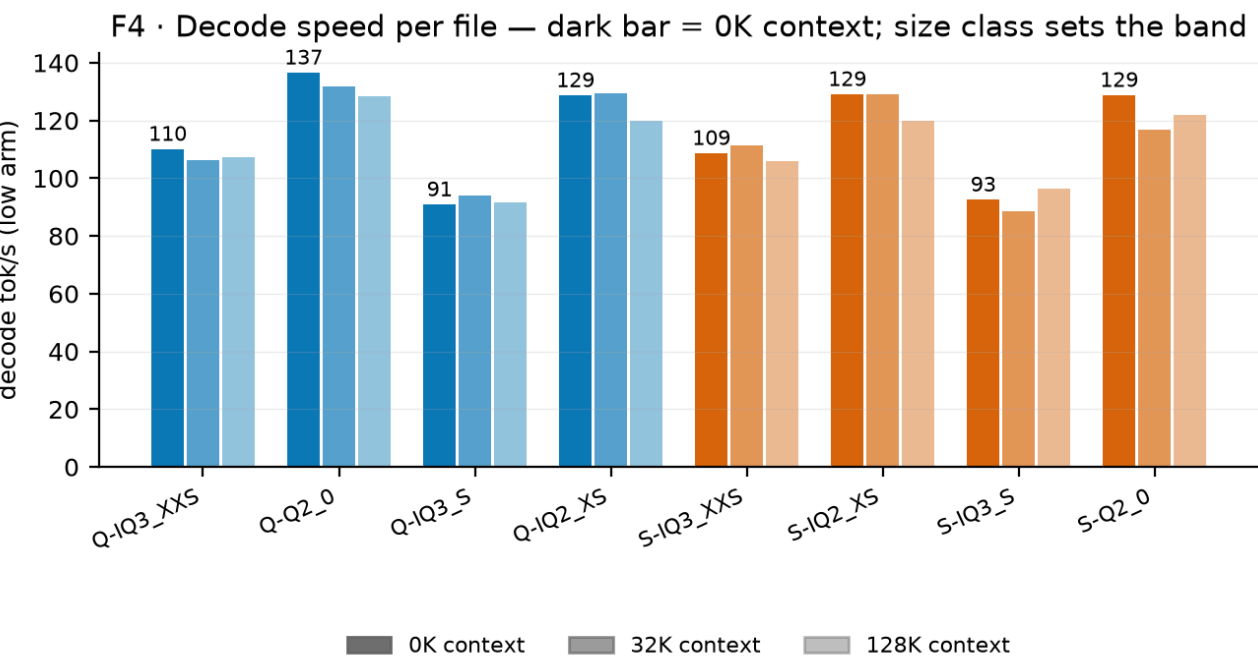


Method: tokens-to-solve & wall time from [Strata](#) run logs (avg_tokens_to_solve, total_time) — raw JSONs in the PR bundle.

SPEED — the size-class rule

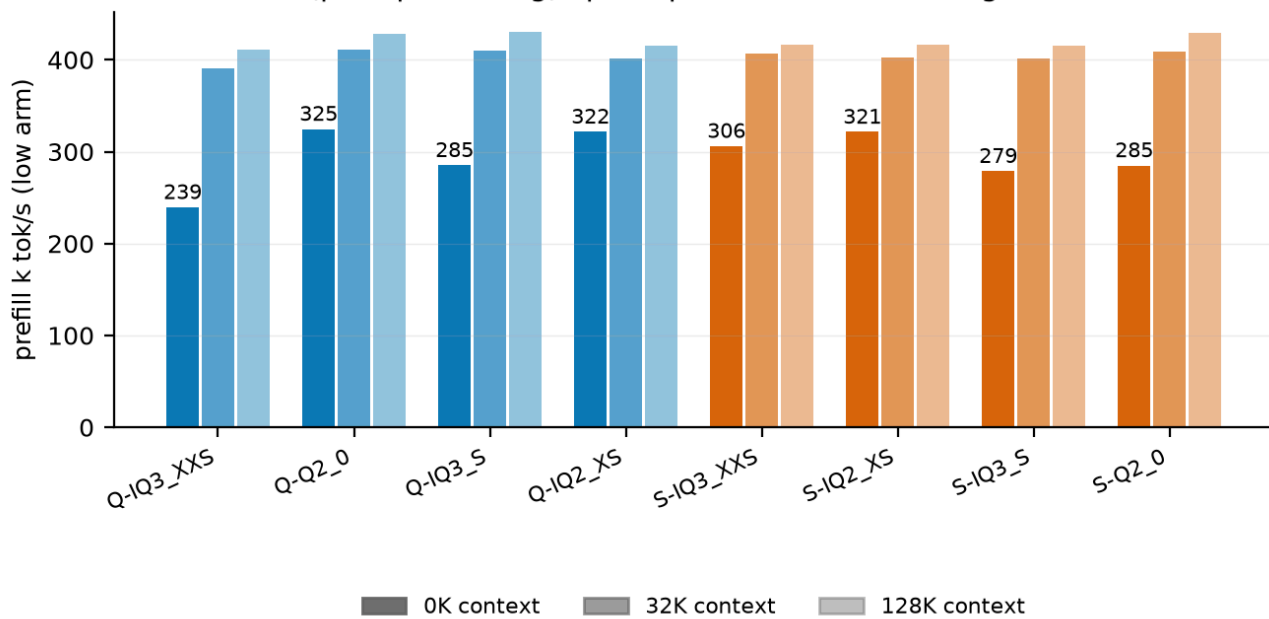
Verdict: decode speed splits by **file size class, not quant name**: 66–68 GB files run 129–137 tok/s, 76–84 GB files run 91–110 on this 16 GB rig. Prefill is huge (239–430K t/s). A 128K context costs up to ~7% decode (one file even got faster).

What these numbers mean: **decode** = tokens/second while answering (what you feel as "speed"). **prefill** = tokens/second while reading your prompt (matters for long documents). Measured by llama-bench at 0K / 32K / 128K context. On a 16 GB card, files bigger than VRAM stream experts from RAM — that's why the 76–84 GB files lose ~35%.



Model (lab)	GB	decode 0K	32K	128K	prefill 0K	32K	128K
Q-IQ3_XXS ISTA-DASLab	75.8	110.1	106.2	107.4	239K	391K	411K
Q-Q2_0 ISTA-DASLab	66.4	136.6 1	131.8	128.4	325K 1	411K	428K
Q-IQ3_S ISTA-DASLab	83.6	90.9	94.0	91.7	285K	410K	430K
Q-IQ2_XS ISTA-DASLab	67.2	128.7	129.4	120.0	322K 2	402K	415K
S-IQ3_XXS UkisAI	76.0	108.7	111.3	106.0	306K	407K	417K
S-IQ2_XS UkisAI	68.2	129.1 2	129.1	119.9	321K 3	402K	417K
S-IQ3_S UkisAI	83.7	92.6	88.5	96.5	279K	401K	415K
S-Q2_0 UkisAI	66.6	128.8 3	116.9	121.9	285K	409K	430K
Gyro-S AgentionAI	58.5	82.4†	68.6†	60.1†	2420†	2753†	2537†

F5 · Prefill (prompt-reading) speed per file — flat and huge across all files



Test: llama-benchy (Strata perf plugin) · Gyro-S: engine-log medians, 2,189 paired samples (peak 134.9 tok/s)

THE FINAL SCOREBOARD — who is good at what

One picture for the whole study. Every arm (file × effort) ranked by an OVERALL score = the average of all seven axes, each scaled 0–100 across the study. Darker cell = better on that axis. The same file at two efforts can land 20 places apart — the effort knob is part of the model you pick. **OVERALL** answers "best all-rounder" — your job may want a specialist; **PICK YOUR FILE** below translates this board into picks.

F15 · Final scoreboard — every arm ranked; darker = better on that axis

		Coding HE+	Agents Pass^5	Hard Mode	MMLU	GSM8K	Debug /25	Decode tok/s	Safety
OVERALL									
S-IQ3_S low	74	96	76	76	85	98	20	93	clean
S-IQ2_XS xhigh	74	94	77	85	83	97	18	129	clean
Q-Q2_0 xhigh	73	95	74	87	78	97	19	137	clean
S-IQ2_XS low	72	95	75	83	83	97	18	129	capped
Q-IQ2_XS low	68	96	74	70	83	97	19	129	capped
Q-IQ2_XS xhigh	64	94	71	80	83	97	19	129	clean
Q-Q2_0 low	60	95	71	74	78	97	19	137	capped
Q-IQ3_XXS xhigh	60	96	73	87	79	99	16	110	capped
Q-IQ3_S xhigh	56	96	68	78	82	97	19	91	capped
S-IQ3_S xhigh	55	94	68	76	85	98	20	93	capped
S-IQ3_XXS xhigh	54	95	73	72	80	96	19	109	capped
S-IQ3_XXS low	54	95	75	67	80	96	19	109	clean
Q-IQ3_S low	54	94	74	76	82	97	19	91	clean
S-Q2_0 low	49	96	78	72	72	90	18	129	clean
Q-IQ3_XXS low	48	95	76	65	79	99	16	110	capped
S-Q2_0 xhigh	37	93	73	83	72	90	18	129	clean
Gyro-S low	n/a	96	n/a	n/a	76	92	16	82	n/a
Gyro-S xhigh	n/a	94	n/a	n/a	76	92	16	82	n/a

ranked by OVERALL (avg of all axes, 0–100)

How to read: rows sorted by OVERALL (top = best all-rounder). **OVERALL averages the axes, so it hides specialists** — Swift Q2_0 low ranks 14th overall yet owns the study's best reliability floor (78.3): a specialist, not a loser. Gyro-S rows show n/a on agent columns: rc1 has no tool support (C1), so its OVERALL is not comparable — that is why it sits at the bottom.

Best per axis

Coding 96.3 — S-IQ3_S low · Q-IQ3_S xhigh · Gyro-S low
Agents floor 78.3 — S-Q2_0 low
Hard Mode 87 — Q-Q2_0 xhigh · Q-IQ3_XXS xhigh
MMLU 85 — S-IQ3_S · GSM8K 99 — Q-IQ3_XXS
Debugging 20/25 — S-IQ3_S · Decode 137 t/s — Q-Q2_0

The three lines that matter

Tied at the top (74): Swift IQ3_S low and Swift IQ2_XS xhigh — the all-rounders.
Effort changes the ranking: Swift IQ3_S is #1 at low, #10 at xhigh. Qwen Q2_0 is #7 at low, #3 at xhigh.
No arm wins everything — and OVERALL is not the ranking to pick by. Agents → floor (S-Q2_0 low: #14 overall, best floor 78.3) · coding → HE+ · budget chat → decode. See PICK YOUR FILE below.

PICK YOUR FILE

Pick by your job, not by OVERALL. The scoreboard crowns all-rounders; these cards crown specialists. Coding is a tie (95.7–96.3 for everyone), so the decision columns are **Pass^5 floor** (score passed in ALL 5 runs) and **Hard Mode** — they predict how a file behaves in your agent stack day after day.

Agent stacks — best reliability floor

Swift Q2_0 (UkisAI) low · #14 overall, best floor

Pass^5 78.3 — highest floor in the study · gap only 4.3 · 66.6 GB, fast decode band
! mid Hard Mode at low (P 74) — go xhigh for 87

Agent stacks — best peak score

Qwen Q2_0 (ISTA-DASLab) xhigh · #3 overall

best single tool score measured (89.4) · best Hard Mode jump 74 → 87 · fast tool walls (8m/trial at low)
! token hog — keep effort low elsewhere

One model for everything

Swift-1.5 IQ3_S (UkisAI) · #1 overall

96.3 HE+ at 15m · best debugger · best MMLU (85) · ties top band
! 83.7 GB → expert-cache on 16 GB VRAM

Fastest code loop

Swift-1.5 IQ3_XXS (UkisAI) · #12 overall

11m56s per 164-task HE+ — study fastest · 95.1
! mid-quant; Swift Q2_0 nearly as fast (12m56s, 95.7)

Single-file simplicity

Gyro-S (AgentionAI) · n/a overall (C1)

96.3 HE+ from one 58.5 GB APR file · own engine
! no tool support in rc1 · engine-log speed only

Knowledge Q&A, modest VRAM

ISTA IQ2_XS (ISTA-DASLab) · #5 overall

97 GSM8K / 83 MMLU at 2-bit size · needle 100%
! weakest debugger of the 2-bit class

Fastest on a 16 GB card

Q-Q2_0 (ISTA-DASLab) / S-IQ2_XS (UkisAI) · ranks #2–#7 overall

the 66–68 GB files decode at 129–137 t/s — the study's top band, measured on the 16 GB RTX 5070 Ti
! reliability gaps as charted above

VRAM: every number in this study was measured on **one card — a 16 GB RTX 5070 Ti**. Files bigger than VRAM stream experts from system RAM (expert cache) — that's why the 76–84 GB files lose ~30%. Bigger cards fit more experts resident and decode faster, but we did not measure 24/32 GB: treat that as expectation, not data.

FOR MAINTAINERS — engine & ops findings

- **C1 · rc1 fork has no tool support** — forced tool_choice ignored, schemas omitted. Biggest product gap found. Gyro-S can't join agentic eval until plumbing lands.
- **C2 · Windows→WSL LAN forwarding resets long/streaming POSTs** — root cause of the "Gyro xhigh crash". Loopback + SSH reverse tunnels clean; 90-line compat proxy is the reference workaround.
- **C3 · --vram-reserve-mib 1212** kills the 0-MiB crash class under WSL contention.
- **C4 · APR/TQ1_0 loads and performs** (96.3 HE+, 92 GSM8K, needle 100% at its 131K cap) — the format works; tool support is the gap, not quality.
- **C5 · Strata 0.1.40.3 held up:** 5 campaigns, zero infra failures after known fixes, pinned digest never drifted, auto-update rollback protected the pin twice.
- **C6 · v4 native_experts mapping is deterministic:** Swift packs 96/96 tensors exact (max |err| 0); ISTA quant 1080/1080 direct, zero conversions.

HOW WE TESTED — AND WHAT WE GOT WRONG FIRST

Method

- One rig, one engine build, identical prompts/seeds/protocols for all 9 files.
- HumanEval+ 164 · tool-eval-bench 92 scenarios × **5 runs per arm** · five-bugs 25+25 · llama-benchy at 0/32/128K · GSM8K/MMLU/IFEval · needle @260K (Gyro-S: 131K serve cap).
- Every table cites its source; raw JSONs, configs and commands ship in the PR.
- Not: a KLD study, vision, multi-seed paper, or claims about GPUs we don't own.

Corrections log (we publish our fixes)

- Single-run tool scores → **n=5** (IQ3_S low 89 → 86.8±0.8).
- Gyro "xhigh crash" reclassified: LAN resets, not the model (clean v2 = 94.4/153/0 timeouts).
- Gyro tool-eval retired: proxy stripped schemas; deeper truth = rc1 has no tools at all.
- "6132 t/s" speed reading was an extraction artifact → engine-log medians (n=2,189).
- Five-bugs early pack mislabel fixed via GGUF headers before the study window.
- **Full claim-by-claim audit (day before publish):** every number in this PDF re-checked against raw JSONs. 12 claims corrected — e.g. safety chips said "capped ★★" (true cap is ★★★, and only 8/16 arms are capped, not all); a hero stat paired two walls that were never equal-quality; "all nine pass needle at 260K" — Gyro-S's serve caps at 131K. Fixed above.

All repos: [evalplus](#) · [tool-eval-bench](#) · [lm-eval-harness](#) · [needle](#) · [Strata](#)

Compiled 2026-10-10 from ALL-METRICS.json — every number machine-extracted, zero hand-typed. Engine pinned 34CDE150B21148E6 · Strata 0.1.40.3. Questions → open an issue on the PR.